

Doc. no.:	P1158R0
Date:	2018-07-11
Audience:	EWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

Concept-defined placeholder types

```
BidirectionalIterator T;  
T it = foo();
```

Motivation

We have three kinds of placeholder types to define variables – `auto` , `decltype(auto)` , and `class_template` . The former two deduce arbitrary types, and the last one deduces specializations. So that we can constrain the deduced type to be a pointer type with `auto*` , or we can constrain the deduced type to be a specialization of `std::iterator` , but we are not able to constrain the deduced type to be an `Iterator` . We need the functionality to deduce types that satisfy the constraints expressed with Concepts.

Introduction

The following example from P0915R0^[1] shows a real issue when missing such a kind of constraints when declaring variables in generic code:

```
template <typename Producer>  
void uploadToGPU(Producer& producer)  
{  
    auto item = producer.next();  
    gpuMemcpy(dst, &item, sizeof(item)); // potentially UB  
}
```

A quick and dirty fix is to use `static_assert` :

```
template <typename Producer>
void uploadToGPU(Producer& producer)
{
    auto item = producer.next();
    static_assert(StandardLayoutType<decltype(item)>);
    gpuMemcpy(dst, &item, sizeof(item));
}
```

which comes with a considerable distance between our idea and the code we written. Here is a different workaround:

```
template <typename Producer>
void uploadToGPU(Producer& producer)
{
    auto item = producer.next();
    StandardLayoutObject(item);
    gpuMemcpy(dst, &item, sizeof(item));
}
```

```
template <StandardLayoutType T>
void StandardLayoutObject(T) {};
```

which is getting close. Look at this τ , it is

- used in place of a type for deduction,
- constrained.

It has all the functionalities we are motivated to add. The only issue is that it is a *constrained-parameter*, declared only in *template-parameter-list*. Can we introduce such a τ in other places?

```
template <typename Producer>
void uploadToGPU(Producer& producer)
{
    StandardLayoutType T;
    T item = producer.next();
    gpuMemcpy(dst, &item, sizeof(item));
}
```

That is the feature we propose – a *constrained-type-name*.

Design Decisions

Model everything after *constrained-parameter*.

1. In a *constrained-type-name*'s scope, the *constrained-type-name* can get involved in deduction multiple times, and must deduce to the same type.

```
template <Iterator T>
void foo(T, std::move_iterator<T>);

Iterator T;
T it = begin(x);
// ...
std::move_iterator<T> i2 = ...;
```

An rvalue reference to *constrained-type-name* is not a forwarding reference, because in

```
template <Copyable T>
void foo(T&&);
```

The `foo` is invented rather than intended. Class template argument deduction can model this intention better:

```
Copyable T;
T &&a = 'a';

template <Copyable T>
struct Foo
{
    Foo(T&&);
};

Foo('a')
```

A *constrained-type-name* is not deduced from a discarded statement in a template entity.

```
template <auto>
auto foo()
{
    Copyable T;
    if constexpr (cond)
        T v = ...;
    else
        T u = ...;
    std::aligned_storage_t<sizeof(T), alignof(T)> s;
    ...
}
```

In the code above, `T` is only deduced from the *true* branch.

2. A *constrained-type-name* is a concrete type in non-deduced context after it has been deduced; if not deduced, the program is ill-formed.

```
template <Copyable T>
void foo(std::array<char, sizeof(T)> a);

foo({ 'a', 'b' }); // ill-formed

Copyable T;
std::array<char, sizeof(T)> a; // ill-formed
```

A *constrained-type-name* is not deduced from a local class scope that is nested to the scope where the *constrained-type-name* is declared.

```
Copyable T;
auto f = [](char* p, size_t sz)
{
    T x = foo(p, sz); // ill-formed if T is not deduced elsewhere
};
```

3. Deducing a *constrained-type-name* must use the copy-initialization syntax. The code that involves *constrained-type-name* cannot be visually distinguished from initializations using concrete types, but we can limit the points of deduction by enforcing an `=` in front of the *initializer-clause*. Meanwhile, it simplifies the semantics because the function parameters are also copy-initialized when deducing the function template parameters.

```
template <Iterator T>
void foo(T);

char s[] = "";
foo(s);

Iterator T;
T p = s;
T np(nullptr); // initializing char*
foo(T(nullptr)); // ok
T ep = nullptr; // ill-formed, T has been deduced
```

4. A *constrained-type-name* should only appear in block scope and have no linkage. In class scopes, there is a complete analysis^[2] to show why we will not have placeholder types on

class members. In namespace scopes, *constrained-type-name* itself will have ODR issues as soon as it gains linkage.

Technical Description

simple-type-specifier:

[...]

auto

decltype-specifier

constrained-type-name

constrained-type-name:

identifier

declaration:

[...]

attribute-declaration

constrained-type-declaration

constrained-type-declaration:

constrained-type-declarator-list ;_

constrained-type-declarator-list:

constrained-type-declarator

constrained-type-declarator-list ,_ *constrained-type-declarator*

constrained-type-declarator:

qualified-concept-name identifier

A *constrained-type-declaration* declares each *identifier* that a *constrained-type-declarator* ends with to be a *constrained-type-name*. A *constrained-type-declaration* shall only appear at block scope.

A *constrained-type-name* is looked up as a *type-name* in its scope. In an initializing declaration of a variable, Let D be a sequence of the *decl-specifiers* that are *type-specifiers* in the *decl-specifier-seq*. If D mentions a *constrained-type-name* defined in the same scope and D followed by the *declarator* can form a function template argument in deduced context, this is a *constrained initializing declaration*, and after each *declarator* there shall be an *initializer-clause* followed by $=$. Let $I_1, I_2, \dots, I_n$ be the list of *declarators*, $E_1, E_2, \dots, E_n$ be those *initializer-clauses* in order, $C_1, C_2, \dots, C_m$ be the *constrained-type-declarators* of all the mentioned *constrained-type-names*. Given the following invented class template,

```
template<  $C_1, C_2, \dots, C_m$  >  
struct f { f(  $D I_1, D I_2, \dots, D I_n$  ); };
```

the *constrained-type-names* are *bounded* to be the types of the template arguments that are deduced from the call $f(E_1, E_2, \dots, E_n)$ as an unevaluated operand. If the class template or the call is ill-formed, the program is ill-formed.

Any use of a *constrained-type-name* refers to the bounded type; if the *constrained-type-name* is not bounded at the point of use or bounded to more than one type, the program is ill-formed.

Examples (not concerning forwarding references):

Given

```
Copyable T;  
Iterator A, Copyable B;
```

for

```
T a[] = "meow";
```

we form

```
template<Copyable T>  
void f(T a[]);
```

Calling `f("meow")` deduces `T` to `char`, so the original declaration becomes

```
char a[] = "meow";
```

For

```
tuple<T*, int> b;  
tuple<A, B> &r = b, c = tuple(a, 3);
```

we form

```
template<Iterator A, Copyable B>  
void f(tuple<A, B> &r, tuple<A, B> c);
```

Calling `f(b, tuple(a, 3))` gives `A = char*` and `B = int`.

Comparing with Other Proposals

There have been a few proposals that can address the motivation of this paper: Concepts TS^[3], in-place syntax^[4], constrained `auto`^[1:1], and YAACD^[5]. The in-place syntax extends the *constrained-type-specifier* from the Concepts TS by allowing optional in-place type names, so I will call their common parts “*constrained-type-specifier*” and discuss the “in-place syntax” separately. YAACD part 1 and constrained `auto` differ only in syntax so that I will group them into “constrained `auto`.” The part 2 makes the syntax compatible with the simple case (a single *decl-specifier* that is a *qualified-concept-name*) of Concepts TS so that I will skip this part.

The *constrained-type-specifier* has two set of rules for deduction, one for simple cases,

```
Copyable &&a = foo();
```

and one for complex cases:

```
tuple<Iterator, Copyable> c = make_tuple(a, 3);
```

In simple cases, deduction result backfills the type for the variable, so `&&` is treated as a forwarding reference; in complex cases, deduction result backfills the template parameters, so `&&` is treated as an rvalue reference. This proposal, “concept-defined placeholder types” do not make this distinction and stick with the latter rule. The other constrained `auto` proposals only handle the simple cases and use the former rule.

The in-place syntax and concept-defined placeholder types can naturally express consistent binding^[6],

```
// in-place syntax
Copyable{T} a = foo();
tuple<Copyable{T}> b = bar();

// this paper
Copyable T;
T a = foo();
tuple<T> b = bar();
```

constrained-type-specifier and constrained `auto` have no such flexibility. The introduced type names can serve other purposes, for example, in

```
Copyable{T} &&a = foo();
```

this `T` can replace `std::remove_cvref_t<decltype(a)>`. The difference is that, in this proposal, introducing these type names are mandatory. By doing so, we solved, simultaneously, the confusion (examples in §3.6^[6:1])

Independent resolution breaks the fundamental equivalence of the notations.

and the dilemma^[7]

We need a way of expressing “same type” for two uses of a concept.
We need a way of saying “different type” for two uses of a concept.

raised in Bjarne's papers.

The concept-defined placeholder types require a specific form of initialization (copy-initialization) to trigger deduction. But it is hard to say whether it is a caveat or a feature, considering that none of the proposals attempt class template argument deduction from partially-specialized template argument lists^[8], which is implied by direct-initialization.

Extensions

If we add multi-argument *constrained-parameters*^[9],

```
template <EqualityComparableWith T U>
void foo(T const& a, U const& b)
{
    if (a == b) // must be valid
```

we should also allow declaring multiple *constrained-type-names* that satisfy a multi-parameter concept at once:

```
EqualityComparableWith T U;
T a = /* ... */;
U b = /* ... */;
if (a == b) // must be valid
```

We may also want to add parameter pack support.

References

1. Romeo, Vittorio, and John Lakos. P0915R0 *Concept-constrained auto*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0915r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0915r0.html>) ↩ ↩
2. Voutilainen, Ville. N3897 *Auto-type members*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3897.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3897.html>) ↩
3. Sutton, Andrew. N4674 *Working Draft, C++ extensions for Concepts*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4674.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4674.pdf>) ↩

4. Sutter, Herb. P0745R1 *Concepts in-place syntax*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0745r1.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0745r1.pdf>) ↩
5. Voutilainen, Ville, et al. P1141R0 *Yet another approach for constrained declarations*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1141r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1141r0.html>) ↩
6. Stroustrup, Bjarne. P0694R0 *Function declarations using concepts*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0694r0.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0694r0.pdf>) ↩ ↩
7. Stroustrup, Bjarne. P0956R0 *Answers to concept syntax suggestions*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0956r0.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0956r0.pdf>) ↩
8. Spertus, Mike. P1021R0 *Extensions to Class Template Argument Deduction*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1021r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1021r0.html>) ↩
9. Yuan, Zhihao. P1157R0 *Multi-argument constrained-parameter*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1157r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1157r0.html>) ↩